

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



II. Busca e Ordenação



II. Busca e Ordenação

1. Métodos de busca em vetores
2. Introdução à ordenação
3. Algoritmos de ordenação por seleção, inserção e troca (bolha)
4. Outros métodos de ordenação
5. Implementação e análise de complexidade



3. Ordenação de Vetores

- Apresentamos os métodos:
 - Seleção
 - Inserção
 - Bolha
- Tinham em comum a complexidade $O(n^2)$
- Outras estratégias para ordenar vetores

4. Outros métodos de ordenação de vetores



4. Outros métodos de ordenação

- **Merge Sort**

- Aplica a estratégia de **Divisão e Conquista**
- Funcionamento:
 1. Divide o vetor em **duas** metades
 2. **Ordena** cada metade **recursivamente**
 3. **Intercala** as metades ordenadas

4. Outros métodos de ordenação

- **Merge Sort**

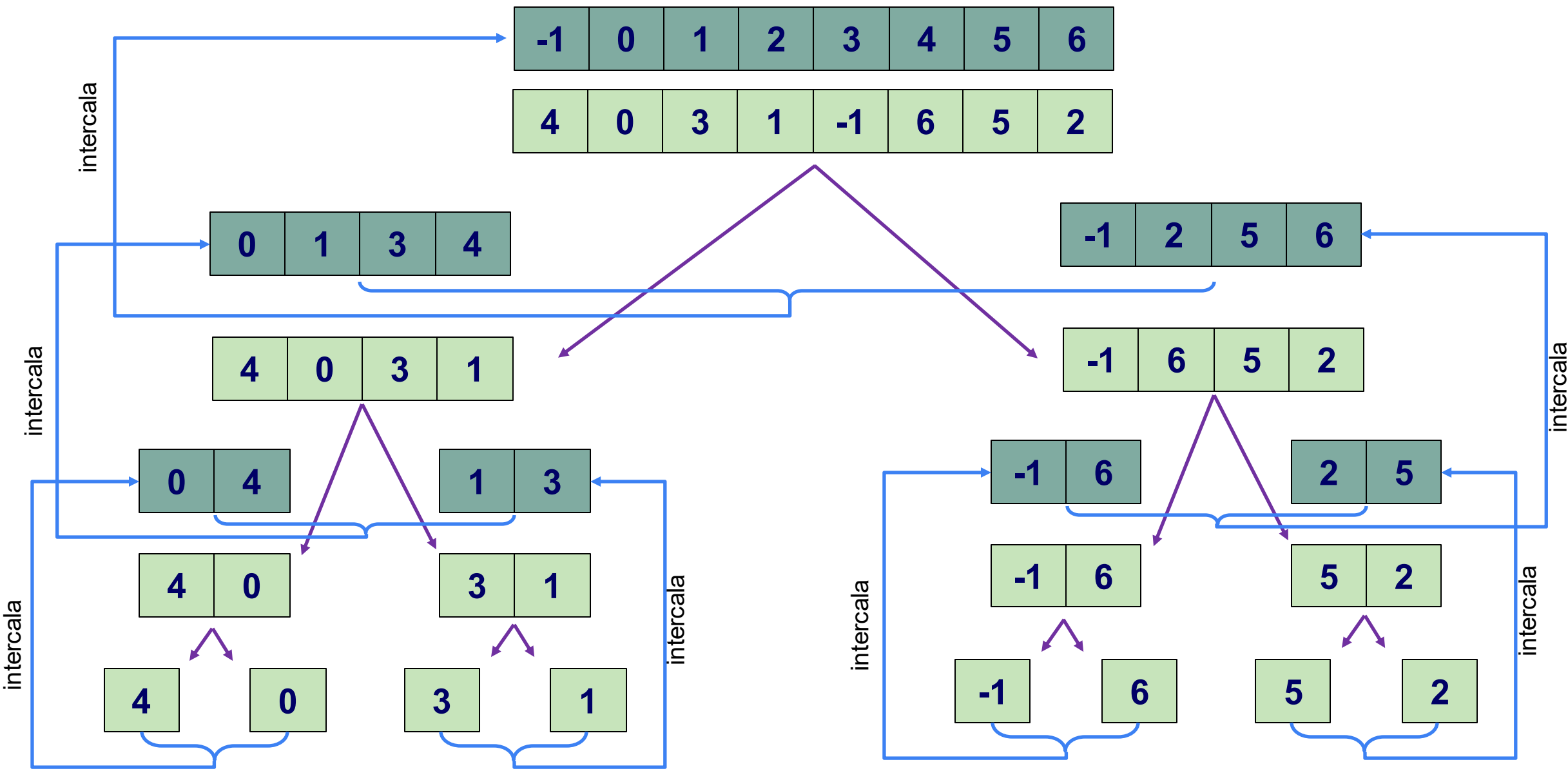
- Algoritmo de ordenação:

```
mergeSort(A, início, fim)
    se início < fim então
        meio ← (início + fim) / 2
        mergeSort(A, início, meio)
        mergeSort(A, meio+1, fim)
        intercalar(A, início, meio, fim)
```



4. Outros métodos de ordenação

- **Merge Sort**
 - Ideia de funcionamento:



4. Outros métodos de ordenação

- Merge Sort

- [outro exemplo](#)
- Observações
 - Estável
 - Requer espaço extra $O(n)$
 - Usado em bibliotecas padrão
- Complexidade
 - Melhor, médio, pior caso: $O(n \log n)$
 - Espaço: $O(n)$



4. Outros métodos de ordenação

- **Quick Sort**

- Aplica a estratégia de **Divisão e Conquista**
- Funcionamento:
 1. Escolhe um **pivô**
 2. **Particiona**: menores à esquerda, maiores à direita
 3. **Ordena** recursivamente cada lado

4. Outros métodos de ordenação

- **Quick Sort**

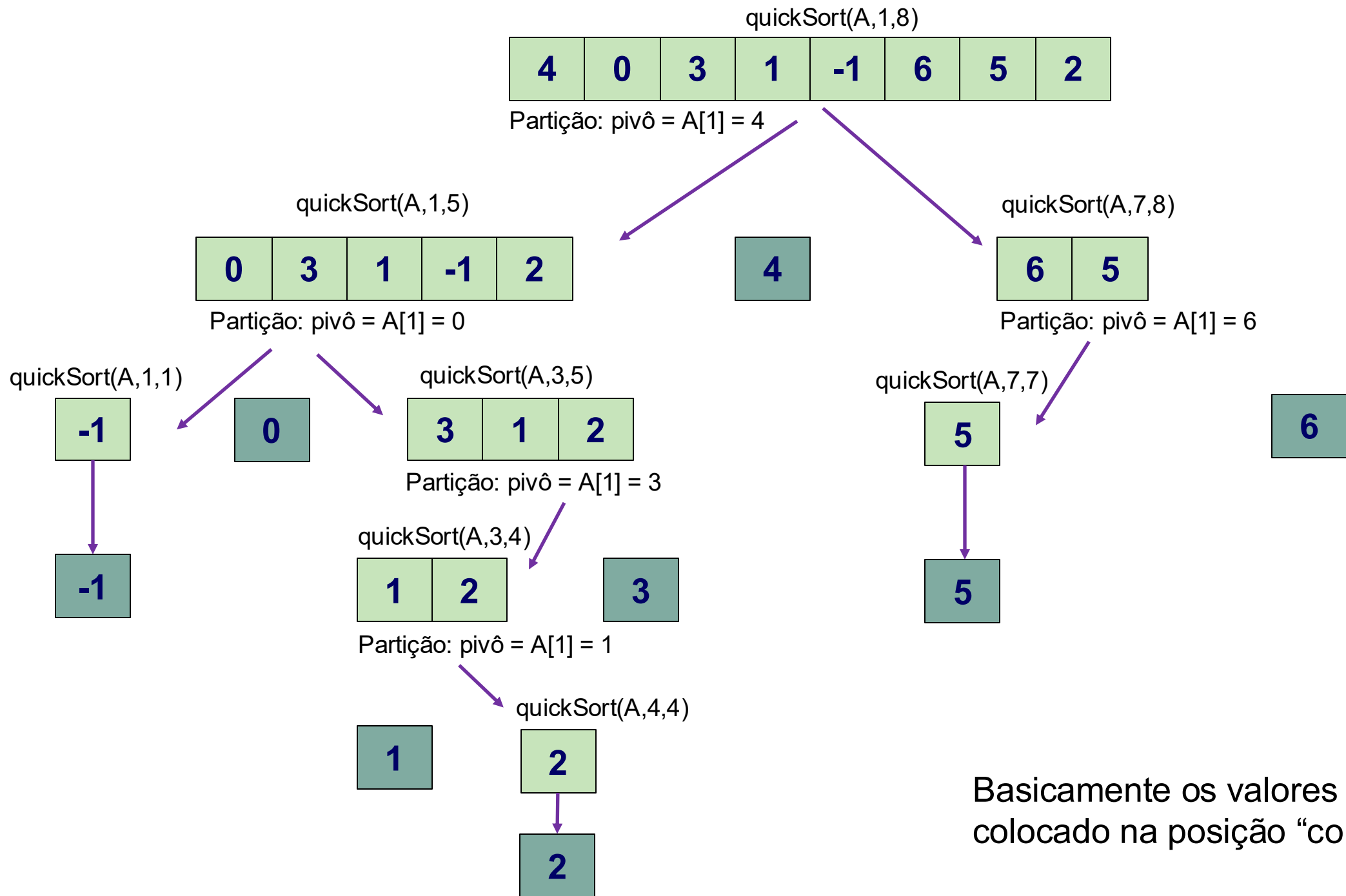
- Algoritmo de ordenação:

```
quickSort(A, início, fim)
    se início < fim então
        p ← particiona(A, início, fim)
        quickSort(A, início, p-1)
        quickSort(A, p+1, fim)
```



4. Outros métodos de ordenação

- **Quick Sort**
 - Ideia de funcionamento:



Basicamente os valores do pivô é colocado na posição “correta” do vetor

4. Outros métodos de ordenação

- Quick Sort

- [outro exemplo](#)
- Observações
 - Muito rápido na prática
 - **Escolha do pivô** influencia o desempenho
 - Não é estável (por padrão)
- Complexidade
 - Melhor: $O(n \log n)$
 - Médio: $O(n \log n)$
 - Pior: $O(n^2)$ (pivôs ruins)
 - Espaço: $O(\log n)$



4. Outros métodos de ordenação

- Quick Sort – desdobramentos de como escolher o pivô
 - Pivô Aleatório (Randomized QuickSort)
 - O pivô é escolhido aleatoriamente entre os elementos do vetor.
 - Reduz a probabilidade de cair no pior caso, mesmo que o vetor já esteja ordenado ou parcialmente ordenado.
 - Complexidade média: $O(n \log n)$



4. Outros métodos de ordenação

- Quick Sort – desdobramentos de como escolher o pivô
 - Mediana de Três (Median of Three)
 - O pivô é escolhido como a mediana entre **três elementos**: o primeiro, o do meio e o último.
 - Isso evita que o pivô seja sempre um extremo em casos de vetor já ordenado.
 - Muito utilizada em implementações práticas de QuickSort.

4. Outros métodos de ordenação

- **Quick Sort – desdobramentos de como escolher o pivô**
 - **Mediana de N (Median of Medians)**
 - Divide o vetor em grupos de 5 elementos, calcula a mediana de cada grupo e depois toma a mediana dessas medianas como pivô.
 - Garante um pivô "bom o suficiente" para sempre manter a divisão razoavelmente balanceada.
 - Complexidade de seleção do pivô: $O(n)$.
 - Usado mais em contextos teóricos ou quando é necessário evitar totalmente o pior caso.



4. Outros métodos de ordenação

- Quick Sort – desdobramentos de como escolher o pivô
 - Amostragem Aleatória (Random Sampling)
 - Seleciona aleatoriamente **k elementos** do vetor e define o pivô como a mediana desses k.
 - Para valores pequenos de k (3, 5, 7), essa técnica equilibra custo de escolha e qualidade do pivô.
 - É um meio-termo entre mediana de três e mediana de n.



4. Outros métodos de ordenação

- **Quick Sort – desdobramentos de como escolher o pivô**
 - **Dual-Pivot QuickSort (Dois Pivôs)**
 - Em vez de um pivô, seleciona-se dois pivôs e divide-se o vetor em **três partes**:
 - menores que o pivô1,
 - entre pivô1 e pivô2,
 - maiores que pivô2.
 - Reduz número de comparações e, em alguns casos, melhora a performance prática.
 - Foi adotado no **Java 7+** como algoritmo padrão para `Arrays.sort()`.

4. Outros métodos de ordenação

- **Quick Sort – desdobramentos de como escolher o pivô**
 - **Dual-Pivot QuickSort (Dois Pivôs)**
 - Em vez de um pivô, seleciona-se dois pivôs e divide-se o vetor em **três partes**:
 - menores que o pivô1,
 - entre pivô1 e pivô2,
 - maiores que pivô2.
 - Reduz número de comparações e, em alguns casos, melhora a performance prática.
 - Foi adotado no **Java 7+** como algoritmo padrão para `Arrays.sort()`.



4. Outros métodos de ordenação

- **Counting Sort (contagem)**
 - Ordena um vetor de **inteiros** ao **contar quantas vezes cada valor aparece** e, em seguida, reconstruir o vetor ordenado com base nessas contagens.
Funciona muito bem quando o intervalo dos valores é **pequeno em relação ao número de elementos**.



4. Outros métodos de ordenação

- **Counting Sort (contagem)**

- **Passos:**

- Determinar o valor máximo do vetor (k).
 - Criar um vetor auxiliar $C[0..k]$ para contar as ocorrências.
 - Preencher $C[i]$ com o número de vezes que i aparece no vetor original.
 - Construir o vetor ordenado B colocando cada elemento $A[j]$ na posição correta.
 - Copiar B de volta para A .

A

4	2	2	8	3	3	1
---	---	---	---	---	---	---

Máximo = 8 → vetor C deve ter 9 posições

C

0	1	2	2	1	0	0	0	1
0	1	2	3	4	5	6	7	8

Percorrer o vetor C adicionando em B

A ideia é 1 vez o 1, 2 vezes o 2, 2 vezes o 3 ...

B

1	2	2	3	3	4	8
0	1	2	3	4	5	6

4. Outros métodos de ordenação

- Utilizando outras técnicas:
 - Counting sort
 - Existe implementação com acumuladores para garantir a estabilidade
 - Complexidade
 - Tempo: $O(n + k)$
 - Espaço: $O(n + k)$
 - Observações:
 - Muito rápido se k não for muito maior que n .
 - **Não é baseado em comparações.**
 - Estável (se implementar a etapa acumulada)



4. Outros métodos de ordenação

- **Radix Sort**

- Ordena números considerando seus **dígitos**, um por vez, do **menos significativo para o mais significativo** (LSD) ou do mais significativo para o menos (MSD).
- Ele usa um algoritmo estável de ordenação **como Counting Sort** em cada dígito.



4. Outros métodos de ordenação

- Radix Sort

- Passos para ordenar

- Determinar o número máximo de dígitos (d) no maior elemento.
 - Para cada posição de dígito i (começando pelo menos significativo):
 - Usar um algoritmo **estável** (geralmente Counting Sort) para ordenar com base nesse dígito.
 - Ao final, o vetor estará ordenado.

A

329	457	657	839	436	720	355
-----	-----	-----	-----	-----	-----	-----

Ordenar pelo primeiro dígito (unidades) - estável

329	457	657	839	436	720	355
-----	-----	-----	-----	-----	-----	-----



A

720	355	436	457	657	329	839
-----	-----	-----	-----	-----	-----	-----

Ordenar pelo segundo dígito (dezenas) - estável

720	355	436	457	657	329	839
-----	-----	-----	-----	-----	-----	-----



A

720	329	436	839	355	457	657
-----	-----	-----	-----	-----	-----	-----

Ordenar pelo terceiro dígito (centenas) - estável

720	329	436	839	355	457	657
-----	-----	-----	-----	-----	-----	-----



A

329	355	436	457	657	720	839
-----	-----	-----	-----	-----	-----	-----



4. Outros métodos de ordenação

- **Radix Sort**

- **Complexidade:**

- Tempo: $O(d \cdot (n + k))$, onde
 - d = número de dígitos
 - k = base usada (ex.: 10 para decimal, 2 para binário)
 - Espaço: $O(n + k)$

- **Observações:**

- Excelente quando os números têm **tamanho fixo limitado**.
 - Usado em aplicações de **CPFs, matrículas, chaves numéricas grandes**.
 - Estável (quando Counting Sort é usado).

4. Outros métodos de ordenação

Comparação dos Métodos

Algoritmo	Tempo Médio	Pior Caso	Espaço	Estável
Inserção	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Seleção	$O(n^2)$	$O(n^2)$	$O(1)$	Não
Bolha	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Não
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(1)$	Não
Counting Sort	$O(n + k)$	$O(n + k)$	$O(k)$	Sim

4. Outros métodos de ordenação

- **Resumo:**

- **Seleção (Selection Sort)**

- Ideia: escolher o menor elemento e colocá-lo na posição correta a cada passo.
 - Aplicação: útil em vetores **pequenos** ou quando as **trocas têm alto custo**, já que faz poucas movimentações.

- **Inserção (Insertion Sort)**

- Ideia: insere cada elemento no lugar correto
 - Aplicação: funciona bem em vetores **quase ordenados** e em conjuntos **pequenos**. Muito usado em sistemas reais como parte de algoritmos híbridos.

- **Bolha (Bubble Sort)**

- Ideia: percorrer o vetor comparando e trocando vizinhos, fazendo as maiores bolhas irem para o fim.
 - Aplicação: **mais didático do que prático**, usado em ensino para introduzir algoritmos de ordenação.

4. Outros métodos de ordenação

- **Resumo:**

- **Merge Sort**

- Ideia: dividir o vetor em metades, ordenar cada parte e depois intercalar.
 - Aplicação: ótimo para **grandes volumes de dados** e quando é necessário manter a **estabilidade**. Muito usado em sistemas que lidam com **arquivos externos**.

- **Quick Sort**

- Ideia: escolher um pivô, particionar e ordenar recursivamente as partes.
 - Aplicação: muito usado na prática, pois é **rápido na média** e bem eficiente em memória interna. É o **padrão em muitas bibliotecas**.

- **Counting Sort**

- Ideia: contar a frequência dos valores e reconstruir o vetor ordenado.
 - Aplicação: usado quando os **valores são inteiros em um intervalo pequeno**. Excelente para dados como **notas, idades, cores de pixels** etc.



4. Outros métodos de ordenação

- **Resumo:**

- **Radix Sort**

- Ideia: ordena os números **dígito por dígito**, usando um algoritmo estável como Counting Sort em cada passo.
 - Aplicação: muito eficiente para ordenar **números grandes** ou **códigos fixos** (ex: CPFs, matrículas, strings de tamanho fixo).

4. Outros métodos de ordenação

Algoritmo	Ideia Principal	Quando Usar
Seleção	Escolhe o menor (ou maior) elemento e coloca na posição correta, repetidamente.	Simples para poucos elementos . Bom para entender a ideia de seleção direta.
Inserção	Insere cada elemento na posição correta dentro da parte já ordenada do vetor.	Útil em vetores pequenos ou quase ordenados.
Bolha	Compara pares vizinhos e troca se estiverem fora de ordem, repetidamente.	Apenas para fins didáticos . Ineficiente em prática real.
MergeSort	Divide recursivamente o vetor em metades, ordena e intercala os resultados.	Bom para grandes vetores . Garante estabilidade .
QuickSort	Escolhe um pivô , particiona elementos menores à esquerda e maiores à direita.	Muito rápido na prática, mas depende da boa escolha do pivô .
CountingSort	Conta a ocorrência de cada valor e reconstrói o vetor ordenado.	Ótimo para valores inteiros em faixa pequena .
RadixSort	Ordena dígito a dígito (da menor posição até a maior), usando Counting como base.	Indicado para números inteiros ou strings grandes . Estável e eficiente.



Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva